Conception phase

Habit Tracker

Benjamin Herrmann, IU14136471, 29.01.2026,

Object Oriented and Functional Programming with Python (DLBDSOOFPP01)

CLI-based habit tracking backend using SQLite persistence and analytics.

1. Goal

The goal of this project is to design and implement a backend system for a habit tracking application in Python. The application allows users to define and track personal habits with different periodicities. For this project daily and weekly periodicities will be used.

By completing habits consistently, users can establish streaks, which serve as a motivation mechanism. Additionally, the system provides analytics such as current streaks, longest streaks, and historical completion statistics.

The focus of the project is on backend logic, data persistence, and analytics, rather than on graphical user Interfaces.

2. System Overview

The system is implemented as a command line interface (CLI) application. The core components of the system are:

- Habit class: Represents a habit with attributes such as name, periodicity and creation date
- Repository layer: Responsible for storing and retrieving habit data and check-off events from a SQLite database
- Analytical module: Implements functional programming techniques to compute streaks and statistical information from stored data
- CLI module: Provides a menu-driven interface that allows users to interact with the system via a terminal.

This modular architecture separates user interaction, data persistence, and analytical logic to improve maintainability and testability.

3. Data Storage and Persistence

Habit data and check-off events will be stored in a SQLite database to ensure persistence between user sessions. Two tables are used:

- Habits table: Stores habit definitions (name, periodicity, creation timestamp)
- Checkoff table: Stores timestamps of completed habits, linked to habits via a foreign key

Using SQLite provides a robust solution compared to simple file storage and enables efficient queries for analytics

4. User Interaction and Application Flow

When the application starts, the user needs to provide a Username on first launch. This username is stored for future sessions. On every subsequent start up, the application displays a personalized greeting and the longest recorded streak.

The main menu provides the following options:

1. Add a new habit (daily or weekly)
2. Check off a habit (record completion)
3. Show current streaks for all habits
4. Show the longest streak for all habits
5. Show rewards (based on streak milestones)
6. Delete an existing habit
7. Exit the application

After executing an action, the user is returned to the main menu until they choose to exit the application.

5. Analytics and Streak Concept

The analytics functionality is implemented using functional programming principles. This means that separate functions are used to process existing habit and check-off data and return results without modifying the stored data. These functions allow the user to view a list of all tracked habits, filter habits by their periodicity, calculate the current streak of a selected habit, determine the longest streak of a specific habit, and identify the longest streak across all habits. Rewards are awarded for predefined streak length (e.g., 7 days for daily habits and 4 weeks for weekly habits).

6. Tools and Technologies

- Programming language and version: Python 3.14.2
- Persistence: SQLite database using Python's sqlite3 module
- Development environment: Visual Studio Code
- Version control: Git and Github
- Testing: pytest unit tests

7. UML Diagram

The diagram shows the CLI module interacting with the Habit Repository for persistence and the Analytics module for streak computation. The Habit class represents the domain model, and the database is accessed through the repository layer.
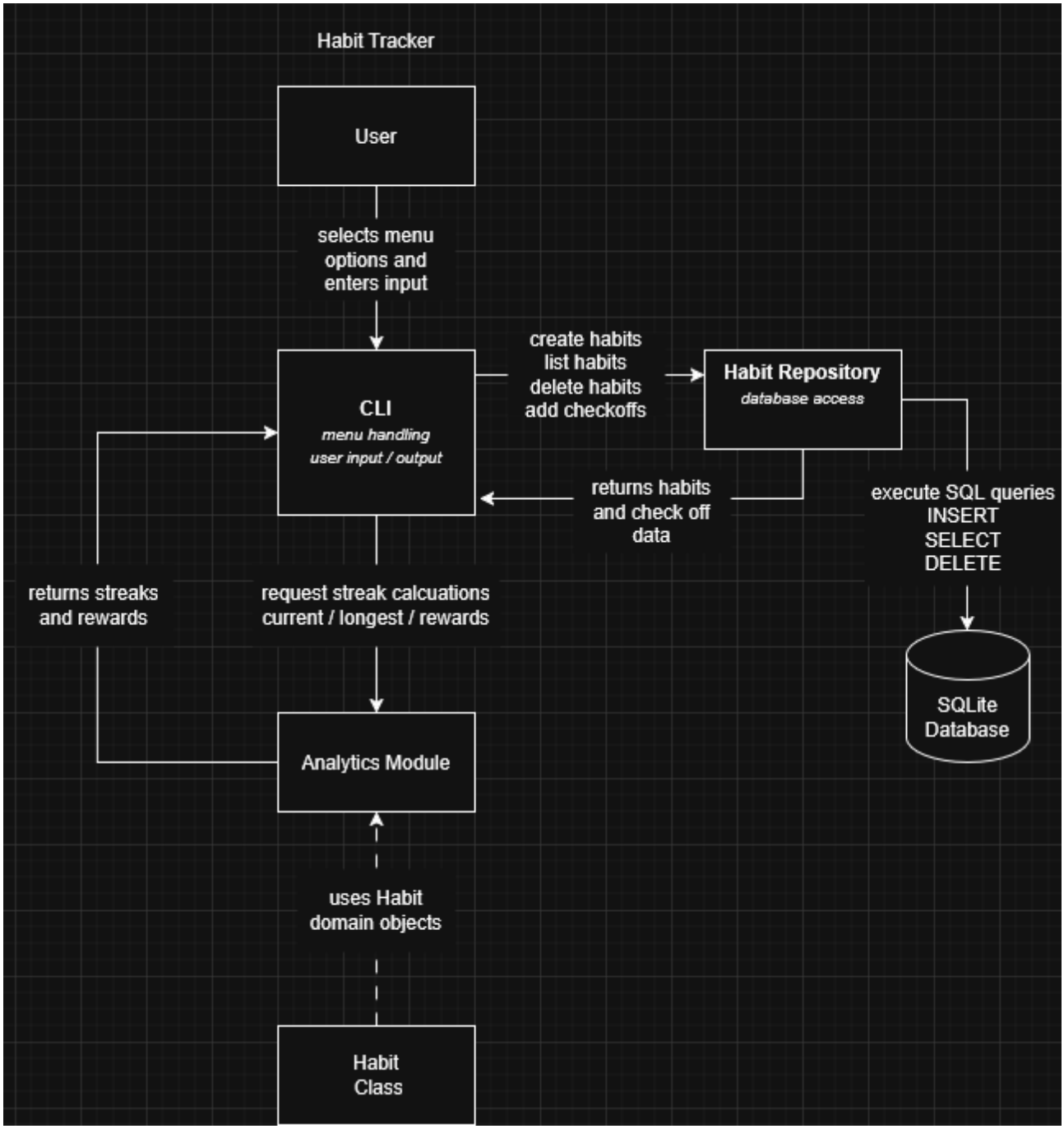
Figure 1: Component interaction of the habit tracker application